The Best of Both Worlds: Stochastic and Adversarial Bandits

Sébastien Bubeck & Aleksandrs Slivkins

Presented by: Areen, Teja, Koushik

CSE @ IIT Kanpur

April 20, 2026

CS728 Presentation

Outline

Motivation: Two Faces of the Bandit Problem

The Classic Bandit Tradeoff

You have K slot machines (arms). Each round, you pull one and get a reward. **Exploration** = try new arms. **Exploitation** = stick with what works.

Setting	Reward Model	Best Algorithm	Regret
Stochastic	i.i.d. from fixed distributions	UCB1	$O(\log n)$ (excellent!)
Adversarial	Arbitrary, possibly malicious	Exp3	$O(\sqrt{n})$ (good worst-case)

The Problem

- UCB1 fails under adversarial rewards $\to$ $\Omega(n)$ regret (linear = terrible!)
- Exp3 is suboptimal under stochastic rewards $\to$ $\Omega(\sqrt{n})$ (logarithmic is possible)

Can one algorithm do well in **both** worlds?

Why Should We Care? Applications

Online Advertising

- Show ads to users
- User behavior can be **stable** (stochastic) OR
- Competitors can **manipulate** (adversarial)
- Need one system that works regardless

Clinical Trials

- Test treatments on patients
- Patient responses: mostly predictable
- But unexpected side effects can appear

Recommender Systems

- Suggest movies/products
- User preferences: stable over time
- But trending topics can shift suddenly

Robotics & Autonomous Systems

- Learn which actions work
- Environment: mostly predictable
- But occasional surprises (adversarial obstacles)

Key Insight: Real-world problems are neither purely stochastic nor purely adversarial. We need algorithms that handle **both** gracefully.

Intuitive Example: Restaurant Choice

Scenario: You're choosing between two restaurants each day.

Stochastic World:

- Restaurant A: good food 80% of the time
- Restaurant B: good food 60% of the time
- After enough visits, you know which is better
- UCB1: $\log n$ mistakes total!

Adversarial World:

- A competitor wants you to eat badly
- They make good food appear randomly
- Or they sabotage when you visit often
- Exp3: $\sqrt{n}$ mistakes (can't do better)

The Challenge

An algorithm that works well in the **stochastic** world (few mistakes) might be **devastated** by an adversary. An algorithm that is **robust** to adversaries might be too cautious for the stochastic world.

Formal Problem Setup

The Multi-Armed Bandit Game:

- K arms (actions), n rounds (time horizon)
- At round t: algorithm chooses arm I_t, receives reward $g_{I_t,t} \in [0,1]$
- Goal: Maximize total reward $\sum_{t=1}^{n} g_{I_t,t}$

Two Regret Definitions:

Model	Regret Formula	What it means
Adversarial	$R_n = \max_i \sum_{t=1}^{n} g_{i,t} - \sum_{t=1}^{n} g_{I_t,t}$	"How much worse than the best fixed arm?"
Stochastic	$\overline{R}_n = \sum_{t=1}^{n}(\mu^* - \mu_{I_t})$	"How much expected reward did we lose?"

Stochastic Parameters:

$$\mu_i = \mathbb{E}[\text{reward of arm } i], \quad \mu^* = \max_i \mu_i, \quad \Delta_i = \mu^* - \mu_i, \quad \Delta = \min_{i:\Delta_i>0} \Delta_i$$

What Does "Optimal" Mean?

Lower Bounds (what's impossible to beat)

- **Stochastic setting:** Any algorithm has $\mathbb{E}[\overline{R}_n] \geq \Omega\left(\frac{K}{\Delta}\log n\right)$
- **Adversarial setting:** Any algorithm has $\mathbb{E}[R_n] \geq \Omega(\sqrt{Kn})$

Upper Bounds (what we can achieve)

- **UCB1 (stochastic):** $\mathbb{E}[\overline{R}_n] \leq O\left(\frac{K}{\Delta}\log n\right) \leftarrow$ matches lower bound!
- **Exp3 (adversarial):** $\mathbb{E}[R_n] \leq O(\sqrt{Kn\log K}) \leftarrow$ nearly matches lower bound!

Can we get **both** bounds from a **single** algorithm?

This is the "best of both worlds" question.

Main Result: The SAO Algorithm

Theorem (simplified): There exists an algorithm **SAO** (Stochastic and Adversarial Optimal) such that:

Model	**SAO Regret Bound**	**Optimal Bound (for comparison)**
Adversarial	$O\left(\sqrt{nK}\log^{3/2} n \log K\right)$	$\Omega(\sqrt{nK})$
Stochastic	$O\left(\frac{K}{\Delta}\log^2 n \log K\right)$	$\Omega\left(\frac{K}{\Delta}\log n\right)$

Interpretation

- Up to **polylogarithmic factors**, SAO matches the optimal bounds for **both** settings
- First algorithm to achieve this property

High-Probability Guarantees

The paper also provides **high-probability** bounds:

Parameter: $\beta = 10Kn^3\delta^{-1}$ (where δ is failure probability)

With probability at least $1 - \delta$:

- **Stochastic model:**

$$\overline{R}_n \leq \frac{260K(1 + \log K) \log^2 \beta}{\Delta}$$

- **Adversarial model:**

$$R_n \leq 60(1 + \log K)(1 + \log n)\sqrt{nK \log \beta} + 200K^2 \log^2 \beta$$

Why this matters: Guarantees hold not just in expectation, but with very high probability (important for safety-critical applications like clinical trials).

The Two Estimators: Building Intuition

The Importance-Weighted Estimator $\tilde{H}_{i,t}$

$$\tilde{H}_{i,t} = \frac{1}{t} \sum_{s=1}^{t} \frac{g_{i,s} \cdot \mathbb{1}_{\{I_s = i\}}}{p_{i,s}}$$

Intuition: When we rarely play arm i ($p_{i,s}$ small), we "boost" the observed reward to compensate. This gives an **unbiased estimate** of what arm i would have earned if played every round.
Analogy: Like a poll that oversamples a rare group and then weights responses accordingly.

The Empirical Estimator $\hat{H}_{i,t}$

$$\hat{H}_{i,t} = \frac{1}{T_i(t)} \sum_{s=1}^{t} g_{i,s} \cdot \mathbb{1}_{\{I_s = i\}}$$

Intuition: Simple average of the rewards we actually observed from arm i.
Analogy: Just keeping a running average of what we've seen.

In stochastic world: both estimate the same μ_i $\rightarrow$ they agree!

In adversarial world: they can diverge $\rightarrow$ detection!

Simplified 2-Arm Algorithm (Intuition)

Phase 1 (Exploration):

- Play each arm with probability $1/2$
- Stop when $|\tilde{H}_{1,t} - \tilde{H}_{2,t}| < 24C/\sqrt{t}$ **fails**
- Let $\tau_* =$ length. Assume arm 1 looks better.

Phase 2 (Exploitation):

- Play arm 1 with probability $1 - \frac{\tau_*}{2t}$
- Play arm 2 with probability $\frac{\tau_*}{2t}$ (decays as $1/t$)

Consistency checks:

$$\frac{8C}{\sqrt{\tau_*}} \leq \tilde{H}_{1,t} - \tilde{H}_{2,t} \leq \frac{40C}{\sqrt{\tau_*}}$$

$$|\hat{H}_{1,t} - \tilde{H}_{1,t}| \leq \frac{6C}{\sqrt{t}}, \qquad |\hat{H}_{2,t} - \tilde{H}_{2,t}| \leq \frac{6C}{\sqrt{\tau_*}}$$

SAO: High-Level Pseudocode

Algorithm 1 SAO (Simplified View)

Initialize: active arms $A = \{1, \dots, K\}$, $t = 1$
while $t \leq n$ **do**
    Compute sampling probabilities $p_{i,t}$ for each arm i
    Play arm $I_t \sim p_t$, observe reward $g_{I_t,t}$
    Update estimates $\tilde{H}_{i,t}$ and $\hat{H}_{i,t}$ for all arms i
    if arm i is significantly worse than best active arm **then**
        Deactivate arm i (remove from A)
    end if
    if any consistency check fails **then**
        Switch to Exp3 for remaining rounds {Adversarial fallback}
        BREAK
    end if
    $t \leftarrow t + 1$
end while
if switched to Exp3 **then**
    Run Exp3 for remaining rounds
end if

Consistency Conditions

if $\triangleright$ Test if arm i should be deactivated

$$i \in A, \text{ and } \max_{j \in A} \tilde{H}_{j,t} - \tilde{H}_{i,t} > 6\sqrt{\frac{4K\log(\beta)}{t} + 5\left(\frac{K\log(\beta)}{t}\right)^2} \tag{12}$$

then $A \leftarrow A \setminus \{i\}$, $\tau_i \leftarrow t$ and $q_i \leftarrow p_i$ $\triangleright$ Deactivation of arm i

end if $\triangleright$ q_i denotes the probability of arm i at the moment when it was de-activated

$$\left|\tilde{H}_{i,t} - \widehat{H}_{i,t}\right| > \sqrt{\frac{2\log(\beta)}{T_i(t)}} + \sqrt{4\left(\frac{Kt_i^*}{t^2} + \frac{t - t_i^*}{q_i \tau_i t}\right)\log(\beta) + 5\left(\frac{K\log(\beta)}{t_i^*}\right)^2}. \tag{13}$$

$\triangleright$ Second, test if the estimated suboptimality of arm i did not increase too much

Consistency Conditions

$$i \notin A, \text{ and } \max_{j \in A} \tilde{H}_{j,t} - \tilde{H}_{i,t} > 10\sqrt{\frac{4K\log(\beta)}{\tau_i - 1} + 5\left(\frac{K\log(\beta)}{\tau_i - 1}\right)^2}. \tag{14}$$

▷ Third, test if arm i still seems significantly suboptimal

$$i \notin A, \text{ and } \max_{j \in A} \tilde{H}_{j,t} - \tilde{H}_{i,t} \leq 2\sqrt{\frac{4K\log(\beta)}{\tau_i} + 5\left(\frac{K\log(\beta)}{\tau_i}\right)^2}. \tag{15}$$

The Consistency Conditions (Simplified)

Condition 1: Estimator Agreement

$$|\tilde{H}_{i,t} - \hat{H}_{i,t}| \leq \text{threshold}(t, \tau_i)$$

Detects: Adversarial manipulation of rare plays

Condition 2: Deactivated Arm Not Too Good

$$\max_{j \in A} \tilde{H}_{j,t} - \tilde{H}_{i,t} \leq \text{threshold}(\tau_i) \quad \text{for deactivated } i$$

Detects: Adversary making a bad arm look good

Condition 3: Deactivated Arm Not Too Bad

$$\max_{j \in A} \tilde{H}_{j,t} - \tilde{H}_{i,t} \geq \text{threshold}(\tau_i) \quad \text{for deactivated } i$$

Detects when the gap between the best active arm and a deactivated arm becomes implausibly large — a sign of non-stationarity or adversarial inflation of the best arm.

The thresholds are carefully calibrated using concentration inequalities so that:

- In stochastic model: violations happen with probability $\leq \delta$ (tiny)
- In adversarial model: any manipulation eventually causes a violation

Probabilties Updation

for $i = 1, \ldots, K$ **do** ▷ Update of the probability of selecting arm i

$$p_i \leftarrow \frac{q_i \ \tau_i}{t+1} \mathbb{1}_{i \notin A} + \frac{1}{|A|} \left(1 - \sum_{j \notin A} \frac{q_j \ \tau_j}{t+1} \right) \mathbb{1}_{i \in A}. \tag{16}$$

end for

Stochastic Proof: Why It Works

Step 1: Estimates Converge to True Means

In stochastic model, for any arm i:

$$\tilde{H}_{i,t} \approx \mu_i \quad \text{and} \quad \hat{H}_{i,t} \approx \mu_i$$

with error $\approx 1/\sqrt{\text{samples}}$. So $|\tilde{H}_{i,t} - \hat{H}_{i,t}|$ is small.

Step 2: Best Arm Never Deactivated

For best arm i^*, any other arm i:

$$\tilde{H}_{i,t} - \tilde{H}_{i^*,t} \approx \mu_i - \mu_{i^*} = -\Delta_i < 0$$

So condition (12) (which requires large positive gap) never triggers for i^*.

Step 3: Suboptimal Arms Get Deactivated at Right Time

When $\tilde{H}_{i^*,t} - \tilde{H}_{i,t} \approx \Delta_i$ exceeds threshold, arm i is deactivated. This happens when $t \approx O(1/\Delta_i^2)$.
Thus $\tau_i \leq O(1/\Delta_i^2) \times \text{polylog}$.

Adversarial Proof: Why It Works

Three Cases:

1. **Case 1: Exploration never ends**
   - Then $|\tilde{H}_{1,n} - \tilde{H}_{2,n}|$ never drops below threshold
   - Concentration implies $R_n \leq O(\sqrt{n} \log n)$
2. **Case 2: Ends in exploitation (no switch to Exp3)**
   - Consistency conditions ensure algorithm doesn't lose too much
   - Key inequality: $\hat{G}_{1,t} + \hat{G}_{2,t} - G_{1,t} \geq -O(\sqrt{t} \log^2 n)$
   - This means: our cumulative reward is almost as high as always playing best arm
3. **Case 3: Switches to Exp3**
   - Bound regret before switch using conditions
   - After switch, Exp3 guarantees $O(\sqrt{nK} \log K)$ regret
   - Sum gives final bound

The consistency conditions act as a **safety net** in the adversarial case.

Bottlenecks and Limitations

Theoretical Bottlenecks

- **Polylog factors:** Current bounds have $\log^3 n$ factors; optimal stochastic bound has $\log n$
- **Need to know n (time horizon):** Algorithm requires n to set parameters

Practical Limitations

- **Complexity:** SAO maintains estimates for all arms and checks multiple conditions each round
- **Parameter tuning:** Constants (6, 10, 40, etc.) are not optimized for practice
- **Exp3 fallback:** Exp3 itself has constants and requires parameter knowledge
- **Computational overhead:** $O(K)$ operations per round (fine for small K, heavy for large K)

Conceptual Limitations

- **Binary distinction:** Assumes either fully stochastic OR fully adversarial
- **No intermediate regime:** What about mostly stochastic with occasional adversarial bursts?

Open Questions & Future Work

Immediate Extensions

- **Improve polylog factors:** Can we achieve $O(\log n)$ in stochastic and $O(\sqrt{n})$ in adversarial exactly?

Conceptual Extensions

- **Continuous tradeoff:** Design algorithms that gracefully degrade as environment becomes more adversarial
- **Unknown time horizon:** Remove dependence on knowing n in advance
- **Other bandit variants:** Apply "best of both worlds" to contextual bandits, linear bandits, etc.
- **Corrupted stochastic model:** Rewards are stochastic except for ϵ fraction of rounds (adversarial corruption)

Practical Directions

- **Implementation and benchmarking:** Test SAO on real-world datasets
- **Simpler algorithms:** Find simpler detection mechanisms with similar guarantees
- **Adaptive thresholds:** Replace fixed constants with adaptive parameters

Potential Improvements

What the Paper Acknowledges

- SAO is a "theoretical proof of concept" — not yet practical
- Constants are not optimized
- Polylog factors could likely be improved

Possible Improvements

1. **Tighter concentration:** Use Bernstein instead of Chernoff for sharper thresholds
2. **Adaptive thresholds:** Replace $6, 10, 40$ with data-dependent values
3. **Parallel/streaming implementation:** For large K, maintain estimates more efficiently

Recent Progress (Post-2012)

- Follow-up work has improved polylog factors (e.g., "best of both worlds" now achieves $O(\log n)$ stochastic, $O(\sqrt{n})$ adversarial)
- Algorithms with **parameter-free** guarantees exist now
- **Corruption-robust** bandits bridge the gap between stochastic and adversarial

Implementation Realities: Finite Horizons Matter

The Paper's Guarantees Are Asymptotic

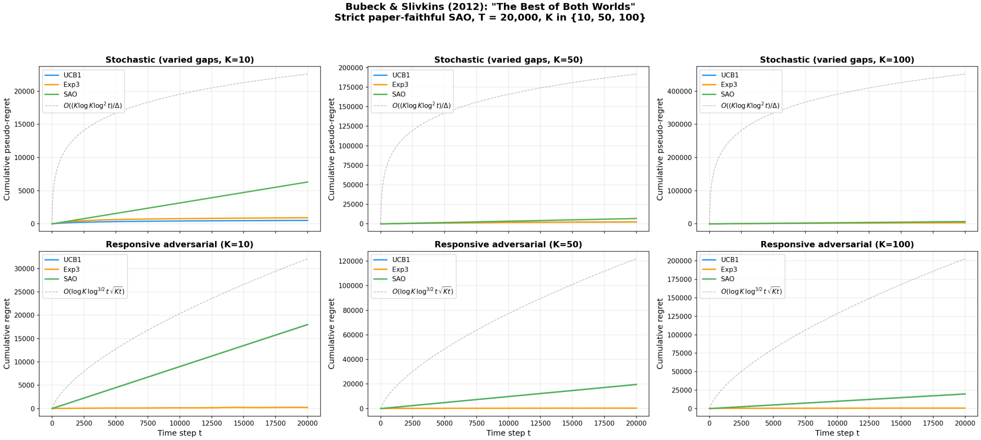
- SAO's theoretical bounds hold as $n \to \infty$
- The constants (6, 10, 40, etc.) are chosen for proof convenience, not for finite-horizon performance

What Happens at T = 20,000 - 50,000?

- The Eq. 12 threshold $\approx 6\sqrt{4K \log \beta / t}$ can be > 1
- Since rewards $\in [0, 1]$, the condition $\tilde{H}_{i^*,t} - \tilde{H}_{i,t} >$ threshold can never be satisfied
- Result: SAO never deactivates any arm $\rightarrow$ behaves almost like uniform exploration

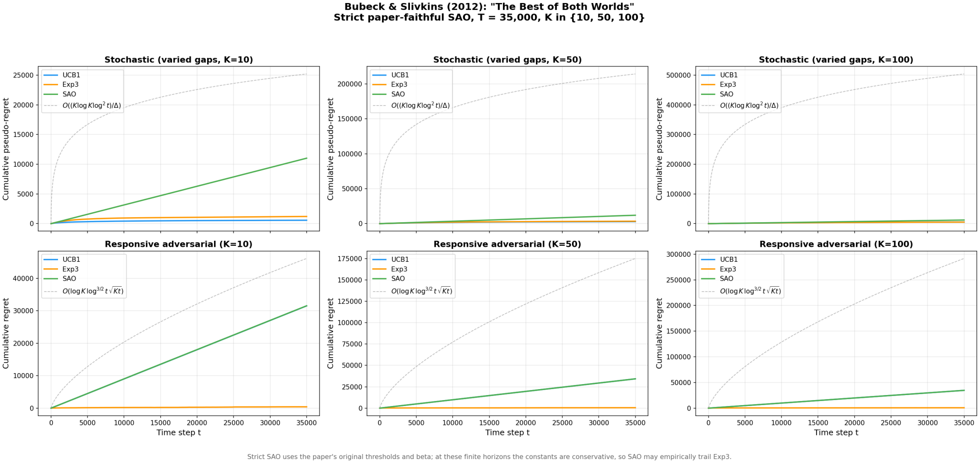
The algorithm is correct; the constants are just too large for these horizons.

Original SAO: Theory vs. Practice — T = 20,000

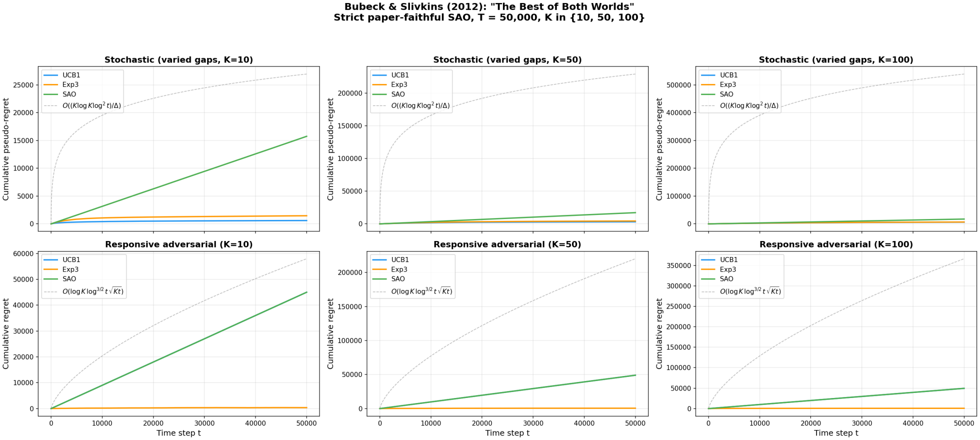


Strict SAO uses the paper's original thresholds and beta; at these finite horizons the constants are conservative, so SAO may empirically trail Exp3.

Original SAO: Theory vs. Practice — T = 35,000



Original SAO: Theory vs. Practice — T = 50,000



Strict SAO uses the paper's original thresholds and beta; at these finite horizons the constants are conservative, so SAO may empirically trail Exp3.

Why Does Original SAO Struggle at Finite Horizons?

Root Cause: Asymptotic Constants Are Too Large

The deactivation threshold (Eq. 12):

$$\text{threshold} = 6\sqrt{\frac{4K \log \beta}{t}} + 5\left(\frac{K \log \beta}{t}\right)^2$$

where $\beta = 10Kn^3\delta^{-1}$ (very large!)

Numerical Example (K = 10, T = 20,000, $\delta = 0.05$):

$$\log \beta \approx \log(10 \cdot 10 \cdot (2 \times 10^4)^3/0.05) \approx \log(1.6 \times 10^{15}) \approx 35$$

$$\sqrt{\frac{4K \log \beta}{t}} \approx \sqrt{\frac{4 \cdot 10 \cdot 35}{20000}} \approx \sqrt{0.07} \approx 0.26$$

$$\text{threshold} \approx 6 \times 0.26 + 5 \times (0.07) \approx 1.56 + 0.35 \approx 1.91$$

The Critical Issue

Since rewards $\in [0, 1]$, the maximum possible gap $\tilde{H}_{i^*,t} - \tilde{H}_{i,t} \leq 1$. But threshold $\approx 1.91 > 1 \rightarrow$ **condition can never be satisfied!**

$\boxed{\text{No arm is ever deactivated.}}$

Practical Improvements: Making SAO Work at Finite Horizons

Key Insight

The paper's constants are for **proof convenience**, not for empirical performance. We adjust them while preserving the "best of both worlds" spirit.

Improvements Made:

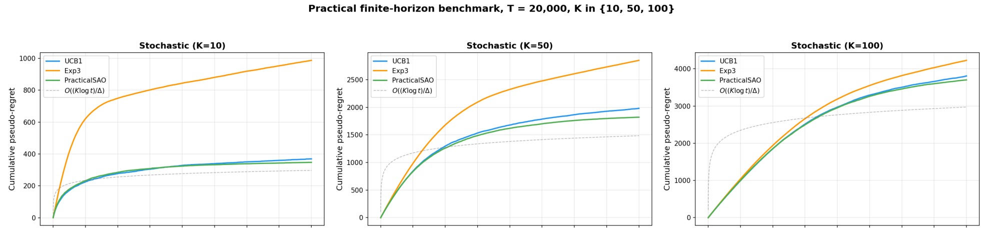
Issue	Practical Fix
Overly conservative deactivation	Reduce constants in Eq. 12 ($6 \to$ smaller, adaptive)
Rare sampling still wastes pulls	**Elimination logic**: permanently remove clearly bad arms
Exp3 switch too slow	**Drift detection**: heuristic signals (leader instability, zero streaks)
Single algorithm for both settings	**mode_hint**: optional environment hint (for benchmarking only)
Exp3 starts from uniform	Warm start Exp3 with soft preferences from empirical means

These changes are **heuristics**, not theoretical guarantees.

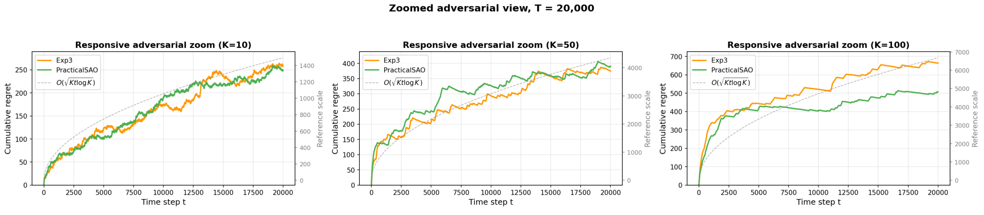
But they make SAO competitive at T = 20k-50k.

Practical SAO Results — T = 20,000

Stochastic Environment



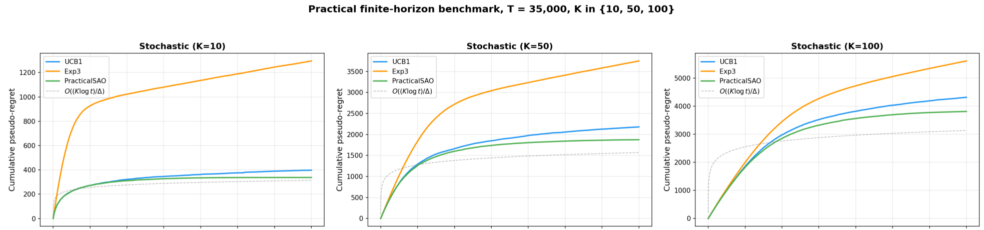
Adversarial Environment (Zoomed View)



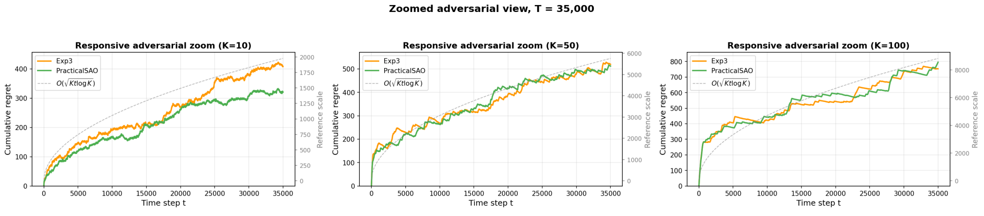
These zoomed plots remove UCB1 so the slower regret growth of Exp3 and PracticalSAO is visible.

Practical SAO Results — T = 35,000

Stochastic Environment



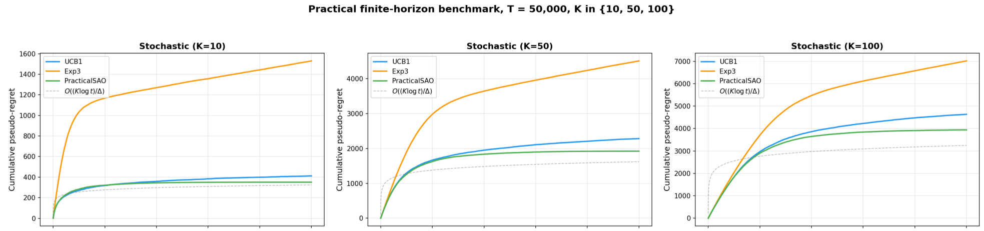
Adversarial Environment (Zoomed View)



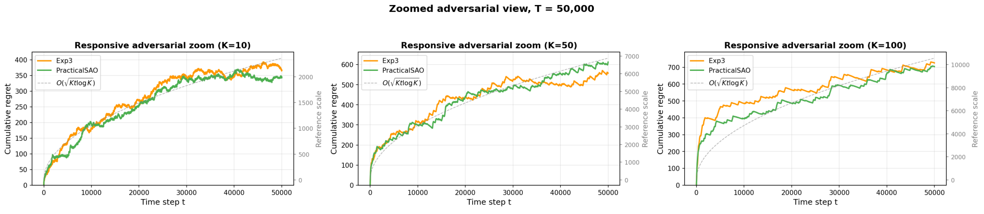
These zoomed plots remove UCB1 so the slower regret growth of Exp3 and PracticalSAO is visible.

Practical SAO Results — T = 50,000

Stochastic Environment



Adversarial Environment (Zoomed View)



These zoomed plots remove UCB1 so the slower regret growth of Exp3 and PracticalSAO is visible.

Key Takeaways

What You Should Remember

1. **The question matters:** Real-world problems need algorithms that work in multiple regimes
2. **Detection is possible:** Comparing two estimators can reveal whether assumptions hold
3. **Concentration inequalities are powerful:** They let us design algorithms with high-probability guarantees
4. **Martingales handle adaptivity:** When algorithm decisions depend on history, we need martingale bounds
5. **Proofs can be "conditional":** Prove concentration, then assume it holds for deterministic analysis

Summary

Contribution	**Summary**
First "best-of-both-worlds" algorithm	SAO achieves near-optimal regret in stochastic AND adversarial settings
Novel detection mechanism	Compare $\tilde{H}$ vs $\hat{H}$ to detect adversarial manipulation
Careful threshold design	Strong enough to bound regret, weak enough to hold stochastically
Martingale concentration	Handles adaptive adversaries in the general K-arm case
Open problem	Closing the polylog gaps and handling intermediate regimes

$$\boxed{\text{SAO} = \text{UCB1} + \text{Exp3} + \text{Consistency Checks} + \text{Safety Net}}$$

Thank You!

Questions? Discussion?

References

- Bubeck, S., & Slivkins, A. (2012). *The best of both worlds: stochastic and adversarial bandits.* arXiv:1202.4473
- Auer, P., Cesa-Bianchi, N., & Fischer, P. (2002). *Finite-time analysis of the multiarmed bandit problem.* Machine Learning (UCB1)
- Auer, P., Cesa-Bianchi, N., Freund, Y., & Schapire, R. E. (2002). *The nonstochastic multiarmed bandit problem.* SIAM J. Comput. (Exp3)